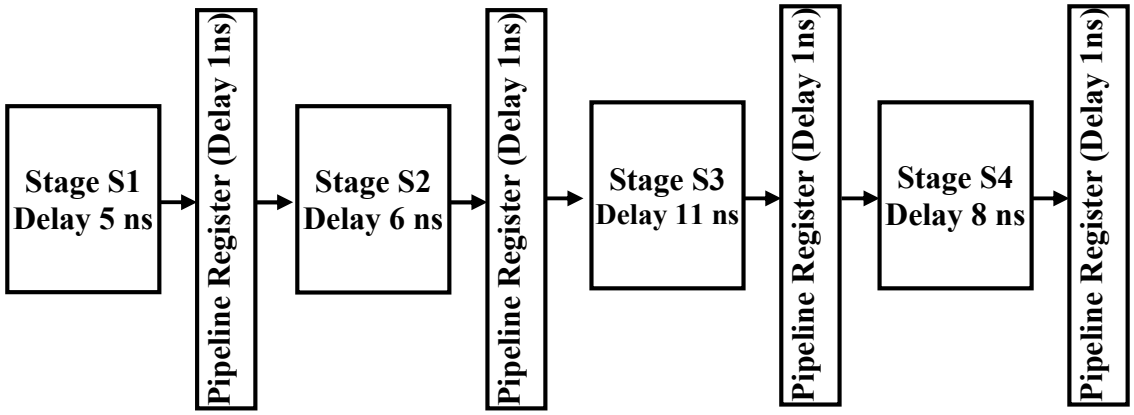
Hint:

$$\textbf{Speedup } (S) = \frac{\text{Non-Pipelined execution time}}{\text{Pipelined execution time}} \text{ (ratio of pipeline to non-pipelined execution)}$$

$$\textbf{Efficiency } (\eta) = \frac{\text{Speedup } (S)}{\text{Number of stages in Pipelined Architecture}}$$

$$\textbf{Throughput} = \frac{\text{Number of executed Instructions}}{\text{Total Time Taken}} \text{ (Nr. of instructions executed/ unit Time)}$$

1. Consider an instruction pipeline with four stages (S1, S2, S3 and S4) each with combinational circuit only. The pipeline registers are required between each stage and at the end of the last stage. Delays for the stages and for the pipeline registers are as given in the figure.



What is the approximate speed up of the pipeline in steady state under ideal conditions when compared to the corresponding non-pipeline implementation?

Solution:

Pipelined clock time tp = max(delay of stages) + 1 = 11 + 1 = 12 ns
None-pipelined clock time = 5 + 6 + 11 + 8 = 30 ns
Speedup = 30/12 = 15/6 = 2.5

2. Consider the following piplined Processors with zero latency pipeline registers.
P1: 4 stage pipeline with stage latencies 1 ns, 2 ns, 2 ns, 1 ns
P2: 4 stage pipeline with stage latencies 1 ns, 1.5 ns, 1.5 ns, 1.5 ns
P3: 5 stage pipeline with stage latencies 0.5 ns, 1 ns, 1 ns, 0.6 ns, 1 ns
P4: 5 stage pipeline with stage latencies 0.5 ns, 0.5 ns, 1 ns, 1 ns, 1.1 ns

Find the clock frequency of each Processor. Which Processor has the highest peak clock frequency?

Solution:

Pipelined clock time tp for P1 = 2 ns & f = 1/2 GHz for P2= 1.5 ns & f = 1/1.5 GHz
for P3 = 1 ns & f = 1 GHz for P4 = 1.1 ns & f = 1/1.1 GHz = 0.9 GHz
The highest peak clock = 1GHz

3. Assuming that individual stages of the datapath have the following latencies:

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

a. If the time for an ALU operation in the above table can be shortened by 25%; Will this affect the **Speedup** obtained from pipelining? If yes, by how much? Otherwise, why?

The speedup according to the latency in the above table is 800/200 = 4

The Speedup of the pipelining will not be affected by shortened the time of the ALU operation by 25% (to be 150 ps), because the clock time remains 200 ps. The speedup in

b. What happen if the ALU operation now takes 25% more time?

If the ALU operation increase by 25% (i.e. the ALU operation delay become 250 ps), this will increase the pipeline clock to 250 ps.
The Speedup of the pipelining will be (200+100+250+200+100)/250 =850/250=3.4
This mean that the speedup will reduce by 0.6/4=0.15 =15%

4. If a pipeline of a new microprocessor is designed to be ideal and a program core with 10^6 instructions. Each instruction takes 100 ps to finish.

a. How long does it take to execute this program on a nonpipelined processor?

This program will take $100*10^6 = 10^8$ ps to finish on a nonpipelined processor.

b. The current state-of-the-art microprocessor has about 20 pipeline stages. Assume it is perfectly pipelined. How much **Speedup** will it achieve compared to the nonpipelined processor?

Since the pipelined is perfect then the speedup is 20 (the number of pipelined stages)

c. Real pipelining isn't perfect, since implementing pipelining introduces some overhead per pipeline stage. Will this overhead affect instruction **Latency**, instruction **Throughput**, or both?

Real pipelining will not affect the *Latency* of the instruction, but the *Throughput*.

5. For the following table, identify the data dependency of the instructions given in the following table. Identify the solution required for each data dependency. Using the 5 stages pipeline of the MIPS processor, show graphically in the second table the pipeline execution of these instructions. Show also the forwarding paths needed to execute these instructions. If forwarding is not available what is the number of cycle needed to execute this program and what is speedup. Is it possible to reorder the instructions to solve stalling the pipeline? If yes, Reorder the program.

I. Nr	Instruction	Data dependancy	Solved by:
1	add $3, $4, $6		
2	sub $5, $3, $7	RAW between I1 & I2 by $3	forwarding
3	lw $7, 100($5)	RAW between I2 & I3 by $5	forwarding
4	add $8, $7, $2	RAW between I3 & I4 by $7	Stalling & forwarding

I. Nr	Instruction	T1	T2	T3	T4	T5	T6	T7	T8	T9
1	add $3, $4, $6	IF	ID	EX	MEM	WB				
2	sub $5, $7, $2		IF	ID	EX	MEM	WB			
3	lw $7, 100($5)			IF	ID	EX	MEM	WB		
4	add $8, $7, $2				⬡	IF	ID	EX	MEM	WB

If forwarding is not available every instruction will be executed alone, Then the number of cycle = 5*4 clocks and the speedup is 20/9 =2.2

6. Consider the following assembly language program:

I1: Move R3, R7	/R3 ← (R7)/	
I2: Load R8, (R3)	/R8 ← Memory (R3)/	**RAW using R3 solved by Forwarding**
I3: Add R3, R3, 4	/R3 ← (R3) + 4/	**WAR using R3 solved by Renaming**
I4: Load R9, (R3)	/R9 ← Memory (R3)/	**RAW using R3 solved by Forwarding**
I5: BLE R8, R9, L3	/Branch if (R9) > (R8)/	**RAW using R9 solved by F. and St.**

This program includes WAW,RAW, and WAR dependencies. Show these dependencies and how to solve them.

7. a. Identify the WAW,RAW, and WAR dependencies in the following instruction sequence and the solution of each type:

I1: R1 = 100	
I2: R1(R5) = R2 + R4	**WAW using R1 By renaming R1 to R5**
I3: R2(R6) = R4 – 25	**WAR using R2 By renaming R2 to R6**
I4: R4(R7) = R1(R5) + R3	**WAW using R4 By renaming R4 to R7**
I5: R1(R5)(R8) = R1(R5)(R8) + 30	**WAR using R1 By renaming R5 to R8**

b. Rename the registers from part (a) to prevent dependency problems.

I1: R1 = 100	
I2: R5 = R2 + R4	**WAW using R1 By renaming R1 to R5**
I3: R6 = R4 – 25	**WAR using R2 By renaming R2 to R6**
I4: R7 = R5 + R3	**WAW using R4 By renaming R4 to R7**
I5: R8 = R8 + 30	**WAR using R1 renamed R5 to R8**

8. For the program shown in the following table, find the RAW, WAW and WAR dependencies and how to solve this dependency. Using the 5 stages pipeline of the MIPS processor, show graphically in the second table the pipeline execution of these instructions. Reorder this program to eliminate or decrease the stalls of the instructions if possible

Instruction	Description	Dependancy	Solution
lw r1, 0(r0)	r1 ← M[r0]		
lw r2, 4(r0)	r2 ← M[r0 + 4]		
add r3, r1,r2	r3 ← r1 + r2	RAW through r2	Solved by Forwarding and stalling
sw r3, 12(r0)	M[r0 + 12] ← r3	RAW through r3	Solved by Forwarding
lw r4, 8(r0)	r4 ← M[r0 + 8]		
add r5, r1,r4	r5 ← r1 + r4	RAW through r4	Solved by Forwarding and stalling
sw r5, 16(r0)	M[r0 + 16] ← r5	RAW through r5	Solved by Forwarding

Nr.	**Instruction**	**T0**	**T1**	**T2**	**T3**	**T4**	**T5**	**T6**	**T7**	**T8**	**T9**	**T10**	**T11**	**T12**
I0	lw r1, 0(r0)	IF	ID	EX	MEM	WB								
I1	lw r2, 4(r0)		IF	ID	EX	MEM	WB							
I2	add r3, r1,r2			⬡	IF	ID	EX	MEM	WB					
I3	sw r3, 12(r0)					IF	ID	EX	MEM	WB				
I4	lw r4, 8(r0)						IF	ID	EX	MEM	WB			
I5	add r5, r1,r4							⬡	IF	ID	EX	MEM	WB	
I6	sw r5, 16(r0)									IF	ID	EX	MEM	WB

The program instructions are reordered as follows to eliminate the stall

lw r1, 0(r0)
lw r2, 4(r0)
lw r4, 8(r0)
add r3, r1, r2
sw r3, 12(r0)
add r5, r1, r4
sw r5, 16(r0)